

Wine Quality Prediction

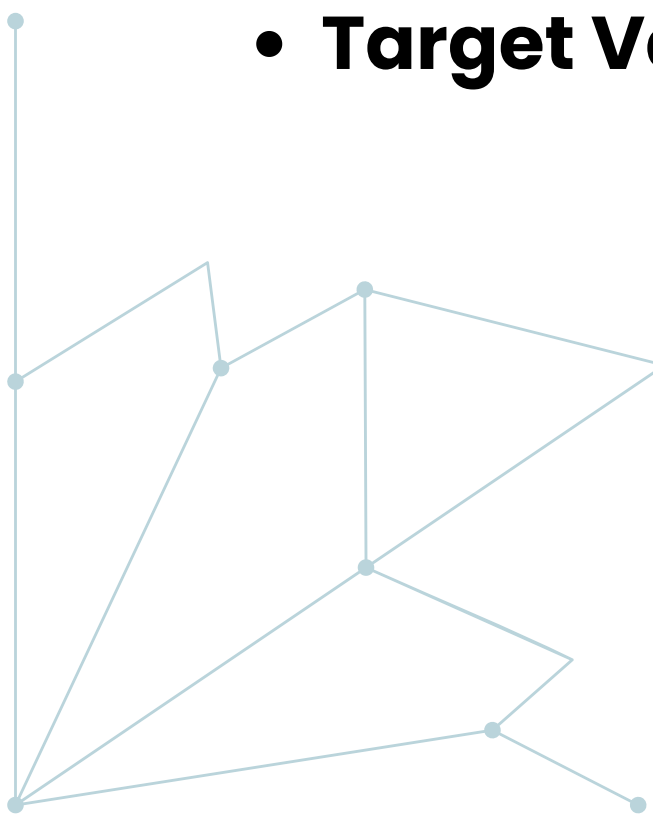
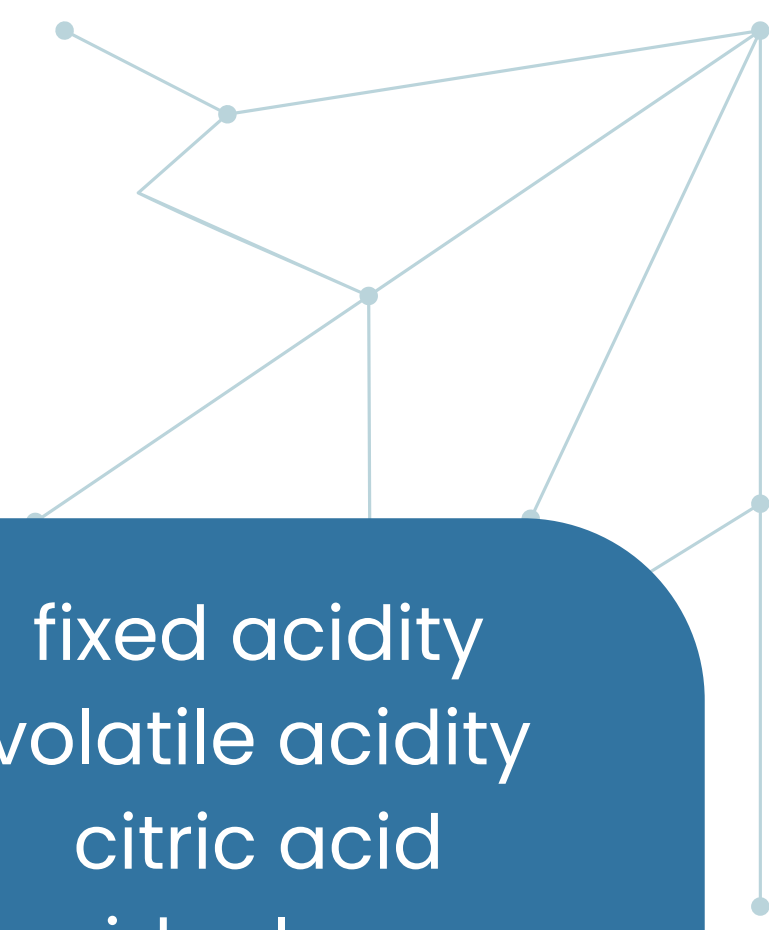
Machine Learning Model

Veronika Angyalová
Digital Transformation Management
veronika.angyalova@studio.unibo.it



Dataset Overview

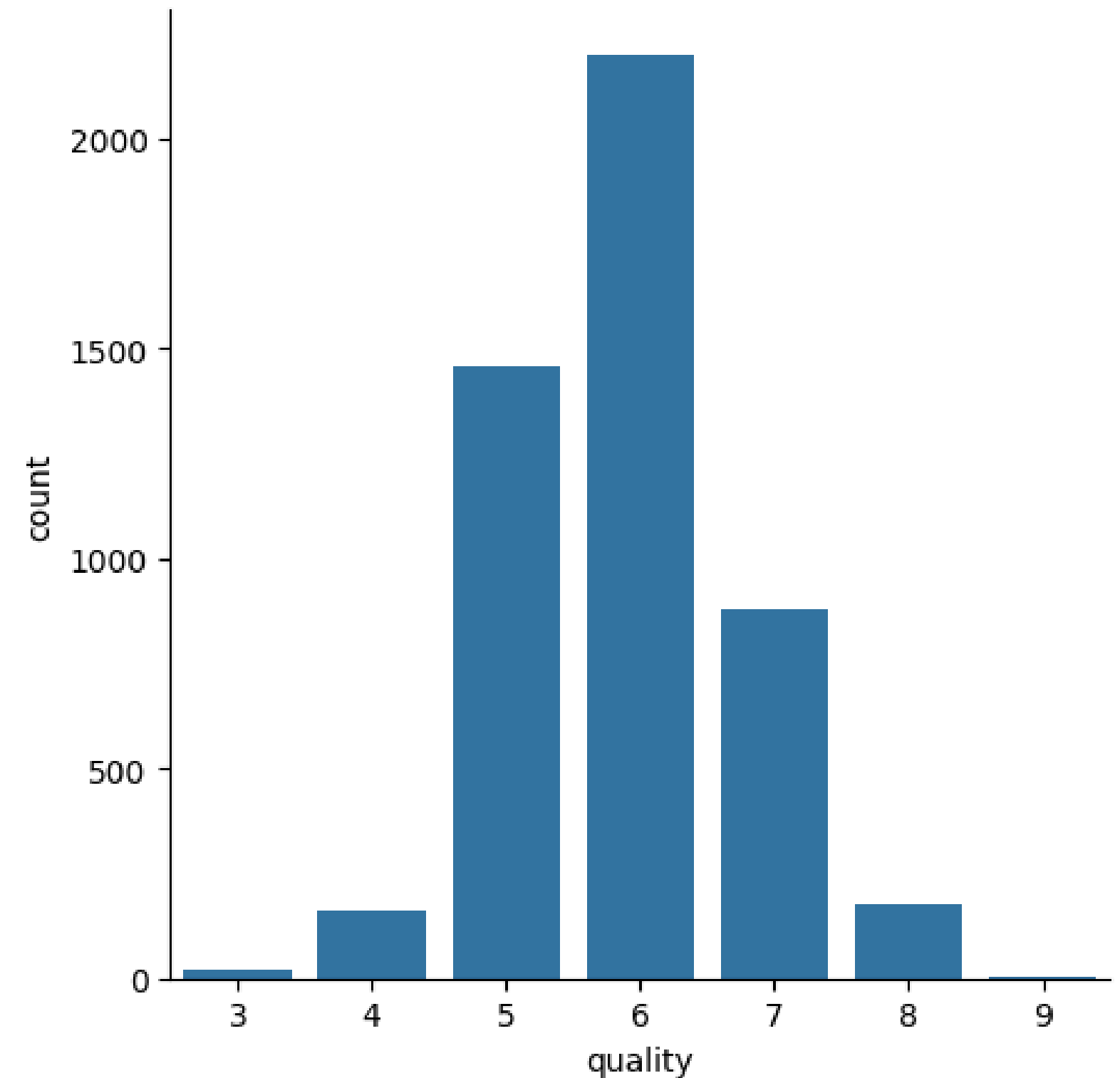
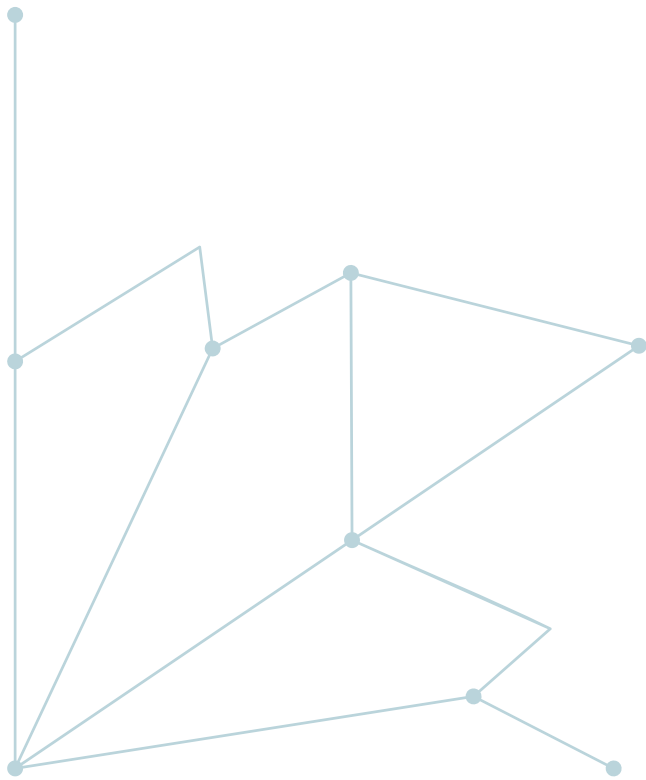
- **Source:** UCI Machine Learning Repository (White Wine Quality).
- **Scale:** 4,898 wine samples.
- **Features:** 11 Physicochemical properties.
- **Target Variable:** Quality scores from 3 to 9.



fixed acidity
volatile acidity
citric acid
residual sugar
chlorides
free sulfur dioxide
total sulfur dioxide
density
pH
sulphates
alcohol
quality

Exploratory Data Analysis

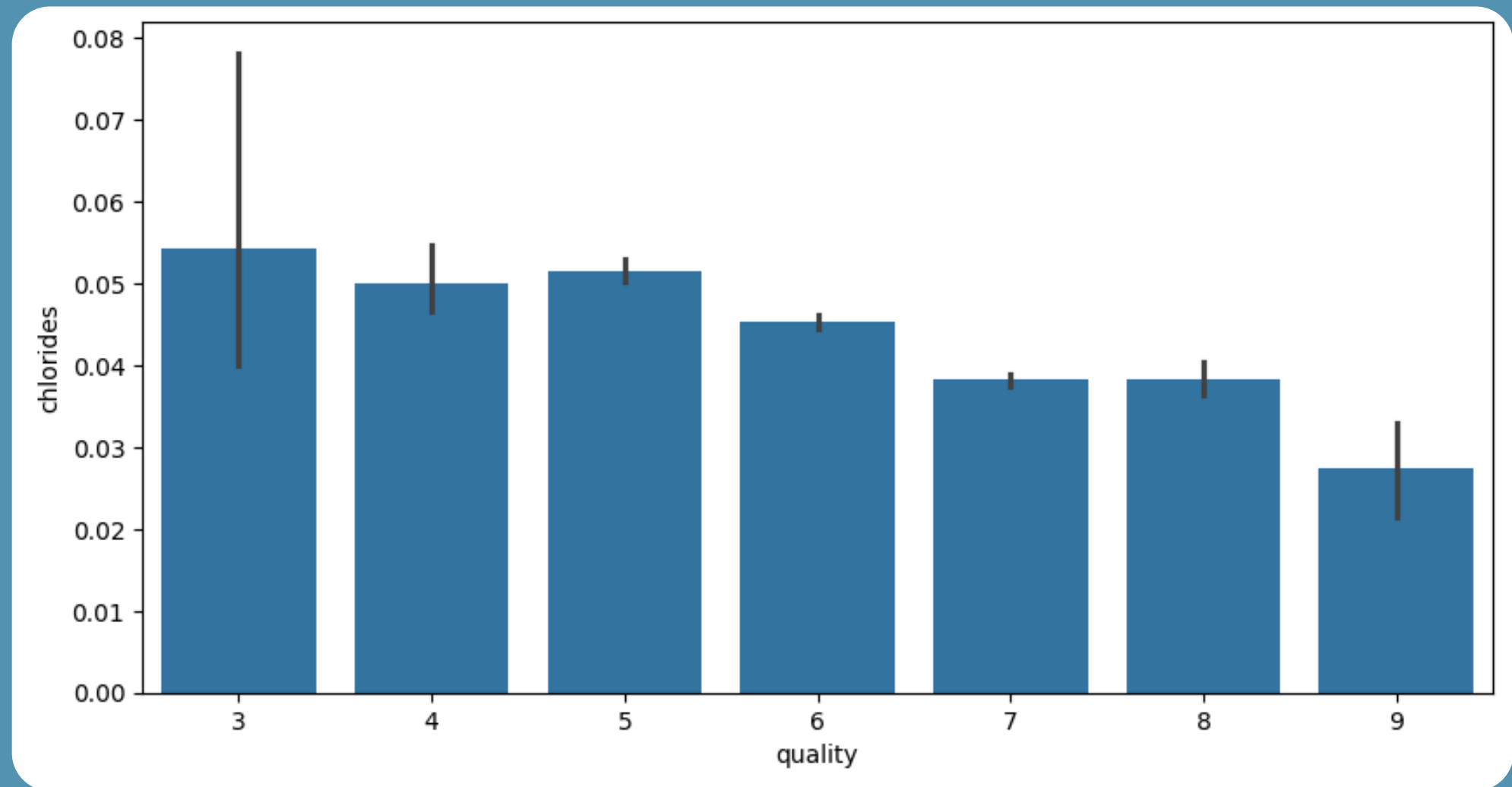
- **Normal Distribution:** Most wines fall in the middle range (scores 5, 6, and 7).
- **Class Imbalance:** Very few extreme scores (3 or 9).



Quality Value Distributions

Dependencies Visualisation

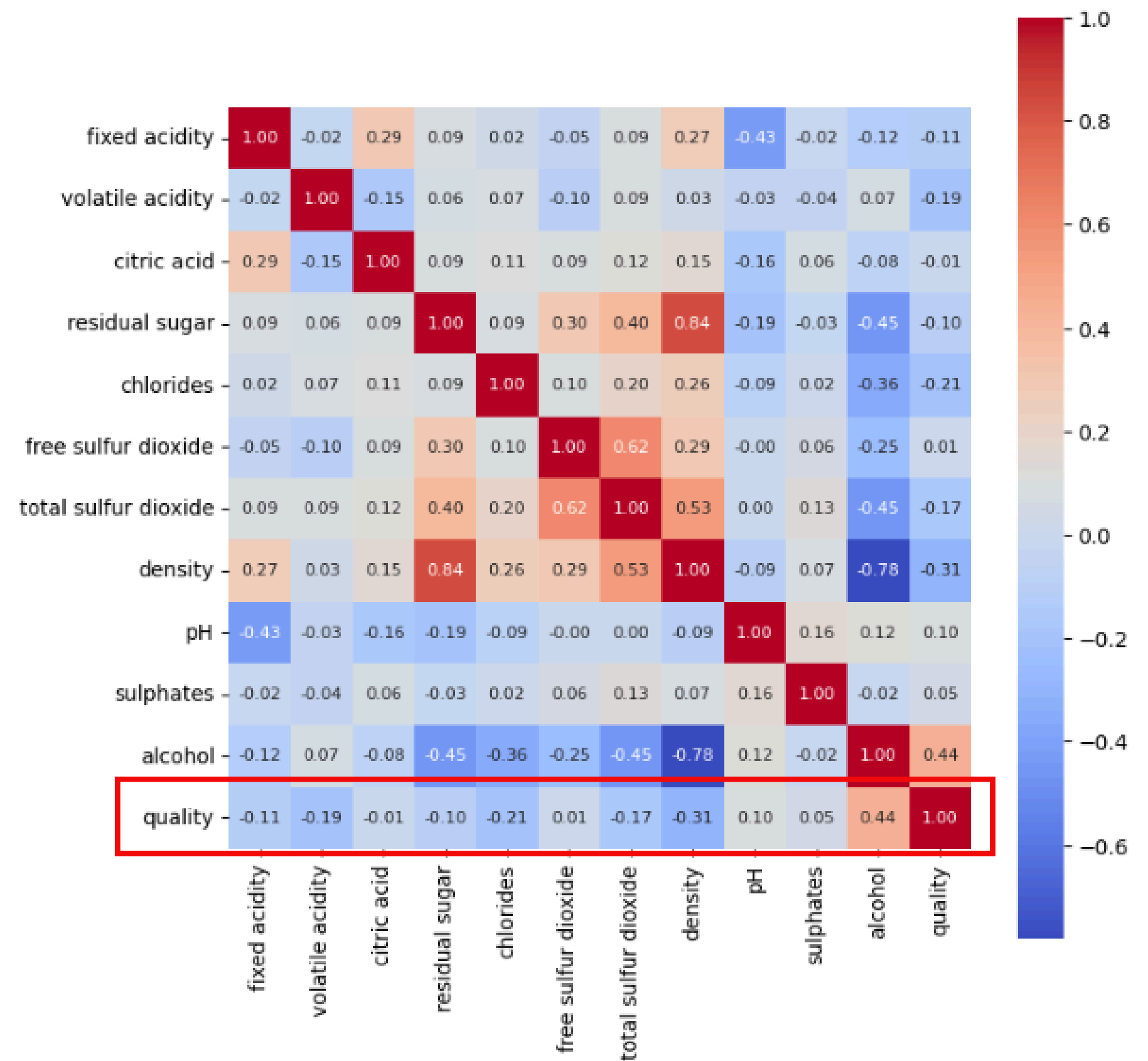
- **Clear Downward Trend:** As chloride levels decrease, quality generally goes up.
- **Chemical Reality:** Lower chlorides mean less "saltiness".



Chlorides vs. Quality

Correlation Heatmap

- **Top Positive Driver:** Alcohol (+0.44) is the strongest indicator of high quality.
- **Top Negative Driver:** Density (−0.31) heavily drags down quality scores.



Corellation Heatmap

Random Forest Classifier

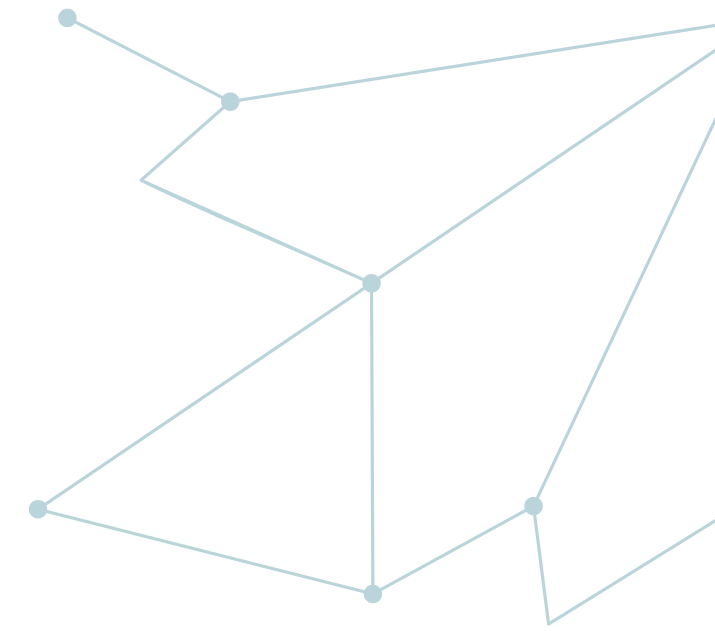
- Binary Classification (Good vs. Bad)
- Multi-class Classification (Good/Average/Bad)

Random Forest Regression

- Exact quality score prediction



Random Forest Classification Models



Data Preprocessing

**20/80
Test-Train Split**

Classifications:

- Binary: ≥ 7 Good, < 7 Bad
- Multi-Class: 6 = Average

Random Forest Algorithm

Algorithm application

Model Evaluation

Model evaluation using Accuracy score:

- Binary: 88.57%
- Multi-Class: 71.12%

Predictive System

Model validation

Binary classification

Model Training Phase: Splitting the dataset and fitting the Random Forest Classifier.

Data Preprocessing

```
X = wine_dataset.drop('quality', axis=1)
Y = wine_dataset['quality'].apply(lambda y_value: 1 if y_value >= 7 else 0)
```

```
X_train_b, X_test_b, Y_train_b, Y_test_b = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```
print(Y.shape, Y_train_b.shape, Y_test_b.shape)
```

```
(4898,) (3918,) (980,)
```

Random Forest Algorithm

```
model = RandomForestClassifier(
    n_estimators=300,
    max_depth=20,
    min_samples_split=5,
    random_state=2
)
model.fit(X_train_b, Y_train_b)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=20, min_samples_split=5, n_estimators=300,
    random_state=2)
```

Model evaluation using Accuracy score

```
X_test_prediction = model.predict(X_test_b)
test_data_accuracy_b = accuracy_score(X_test_prediction, Y_test_b)
```

```
print('Accuracy:', test_data_accuracy_b)
```

```
Accuracy: 0.8857142857142857
```


Prediction Models

Binary Classification

```
# quality = 8 => good quality wine
input_data = (6.7, 0.23, 0.31, 2.1, 0.046, 30.0, 96.0, 0.9926, 3.33, 0.64, 10.7)

input_data_as_numpy_array = np.asarray(input_data)
input_data_resaped = input_data_as_numpy_array.reshape(1,-1)
prediction = model.predict(input_data_resaped)
print(prediction)

if (prediction[0]==1):
    print('Good quality wine')
else:
    print('Bad quality wine')
```

[1]
Good quality wine

Multi-Class Classification

Real-World Simulation: Feeding an unlabelled chemical profile into the system to predict its category.

```
# quality = 6 => average quality wine
input_data = (8.1, 0.28, 0.40, 6.9, 0.050, 30.0, 97.0, 0.9951, 3.26, 0.44, 10.1)

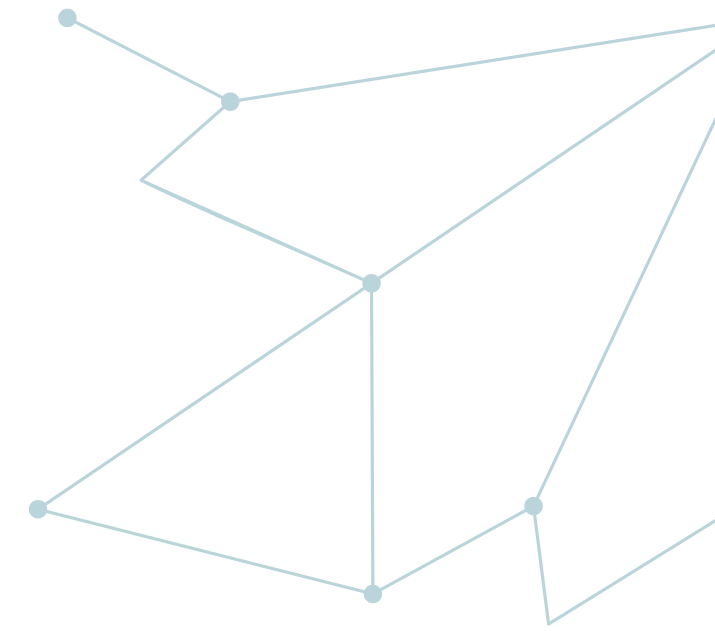
input_data_as_numpy_array = np.asarray(input_data)
input_data_resaped = input_data_as_numpy_array.reshape(1,-1)

prediction = model_m.predict(input_data_resaped)
print(prediction)

if (prediction[0]==0):
    print('Bad quality wine')
elif (prediction[0]==1):
    print('Average quality wine')
else:
    print('Good quality wine')
```

[1]
Average quality wine

Random Forest Regression Model



Data Preprocessing

**20/80
Test-Train Split**

Random Forest Alghoritm

**Alghoritm
aplication**

Model Evaluation

**Model evaluation using
MSE and R2:**

- **MSE: 0.399**
- **R2: 0.486**

Predictive System

Model validation

Regression

Regression Setup: Applying the Random Forest Regressor for continuous score prediction.

Random Forest Regression

```
X_reg = wine_dataset.drop('quality', axis=1)
Y_reg = wine_dataset['quality']
```

```
X_train_r, X_test_r, Y_train_r, Y_test_r = train_test_split(X_reg, Y_reg, test_size=0.2, random_state=2)
```

```
reg_model = RandomForestRegressor(
    n_estimators=500,
    max_depth=30,
    min_samples_split=2,
    random_state=2
)
reg_model.fit(X_train_r, Y_train_r)
```

```
RandomForestRegressor(
    max_depth=30, n_estimators=500, random_state=2
)
```

```
reg_predictions = reg_model.predict(X_test_r)
```

```
mse = mean_squared_error(Y_test_r, reg_predictions)
r2 = r2_score(Y_test_r, reg_predictions)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 0.3991321061322077
R2 Score: 0.4858122214391003
```

Predictive model

Exact Score Prediction:

Outputting a specific numerical value based on the chemical inputs

```
# quality = 4
input_data = (7.6, 0.48, 0.37, 0.8, 0.037, 4, 100, 0.9902, 3.03, 0.39, 11.4)

input_data_as_numpy_array = np.asarray(input_data)
input_data_reshaped = input_data_as_numpy_array.reshape(1, -1)
prediction = reg_model.predict(input_data_reshaped)

print("Predicted Score:", prediction[0])

rounded_prediction = round(prediction[0])
print("Rounded Prediction:", int(rounded_prediction))
```

```
Predicted Score: 4.284
Rounded Prediction: 4
```

Thank you



veronika.angyalova@studio.unibo.it
<https://github.com/veronikaangyal/Wine-Quality-Prediction-ML>